

OBJECTIVE-PRESERVING, OPTIMIZER-AWARE TASK SAMPLING FOR MULTI-TASK LLM POST-TRAINING

Anonymous authors

Paper under double-blind review

ABSTRACT

Multi-task LLM post-training makes two separate decisions. Teacher-loss weights define the optimization target, while task probabilities allocate rollout and teacher compute. Using one mixture for both decisions changes the target whenever the task schedule changes.

We study systems that draw one task before each task-homogeneous update. The sampled gradient receives its target weight divided by its sampling probability. Standard AdamW squares this factor in its second-moment update, so the optimizer state changes with the sampling proposal. We replace that square with a single factor. This one-line change makes the expected first- and second-moment changes match the baseline whose task probabilities equal the target weights. Gradient-Preserving Adaptive Sampling (GPAS) then measures task gradient scale in the lagged AdamW metric and samples in proportion to target weight times measured scale. Cost-GPAS also divides by the square root of measured task time. The cost rule is the one-draw form of cost-constrained Neyman allocation.

In a controlled three-task study, GPAS reduces AdamW-metric estimator error to 0.697 of uniform sampling. Raw gradient-norm sampling reaches 7.178 times the uniform error in the same metric. A second controlled test verifies the AdamW moment correction under a proposal different from the target mixture. GPAS stores one scale and one runtime estimate per task and reuses the gradient and timing produced by each training step.

1 INTRODUCTION

Multi-task post-training spends very different amounts of optimization compute on tasks that appear equally often. In Open-MOPD, instruction-following data account for 20.3% of prompts and only 0.99% of gradient tokens. Early per-token reward magnitudes also differ by a factor of 4.9 (Gao et al., 2026). A nominal data mixture can therefore provide a poor description of the updates received by each task.

We separate two variables that are commonly tied together. Target weights λ define a teacher-loss objective

$$F(\theta) = \sum_i \lambda_i L_i(\theta). \quad (1)$$

Sampling probabilities q decide which task receives the next rollout, teacher call, and optimizer step. The teacher-loss weights describe the chosen training objective. Capability priorities require separate benchmark evaluation because one teacher can move performance on several domains (Li et al., 2026). Keeping λ and q separate lets an experiment change compute allocation while holding the teacher-loss objective fixed.

Our execution regime selects one task before generation and forms a task-homogeneous batch. This regime matches a worker that keeps one teacher in memory or calls one task-specific reward service. Generation and teacher evaluation account for most of the step cost. Systems that can serve several tasks in one step can use a mixed batch; Open-MOPD’s token-share weighting is a corresponding design for that regime. We focus on one-task sampling.

If task i has target weight λ_i and proposal probability q_i , the factor λ_i/q_i gives an unbiased target-gradient estimate. Standard AdamW also squares this factor when it updates the second mo-

ment. The expected second-moment observation then contains λ_i^2/q_i and changes with the proposal (Lahire, 2023). The sampler and the AdamW metric become coupled.

We make one change to AdamW: its second-moment observation uses $(\lambda_i/q_i)G_i^2$. Standard AdamW uses the square of the corrected gradient. The expected first- and second-moment changes now match the baseline $q = \lambda$. GPAS uses the lagged AdamW preconditioner to measure each task’s update scale s_i and selects $q_i \propto \lambda_i s_i$. If task i costs c_i seconds, Cost-GPAS selects $q_i \propto \lambda_i s_i / \sqrt{c_i}$, the cost-constrained Neyman allocation for one task draw.

Our evaluation follows the same separation. Fixed- λ runs compare uniform sampling, raw gradient-norm sampling, GPAS, Cost-GPAS, and a corrected D^3 proposal. Two additional D^3 runs share a time-varying target and isolate the benefit of GPAS allocation on top of a curriculum. A standard AdamW arm isolates the moment correction. Teacher loss is the primary metric; capability benchmarks provide a secondary endpoint.

The paper makes three contributions:

- We give a one-task-per-step protocol that keeps the teacher-loss objective fixed for any proposal and preserves the expected changes in both AdamW moments.
- We derive the optimizer-aware proposal $q_i \propto \lambda_i s_i / \sqrt{c_i}$ from a cost-constrained sampling objective. The identity metric recovers classical gradient-norm sampling.
- We define a matched MOPD comparison that measures target changes and compute allocation separately. The design also measures their composition.

Section 3 defines the objective and optimizer update. Section 4 gives the allocation rule. Section 5 connects each claim to a controlled or MOPD measurement.

2 RELATED WORK

Changing the target and changing the allocation. Multi-task methods often change loss weights or gradient directions to improve a capability trade-off (Sener & Koltun, 2019; Yu et al., 2020; Liu et al., 2024; Chen et al., 2018). DRATS, D^3 -MOPD, and Open-MOPD’s gap-following schedule use task progress to change the training target (Corrado et al., 2026; Sun et al., 2026; Gao et al., 2026). Importance-sampling methods keep a fixed target and change example frequency to reduce gradient error or compute (Alain et al., 2015; Katharopoulos & Fleuret, 2018; Mohri et al., 2026). GPAS is in the second group. A curriculum can supply a time-varying target first, after which GPAS allocates task draws for that target.

Multi-teacher post-training. MOPD routes student rollouts to task teachers and trains a shared student with dense token feedback (Ma et al., 2026). Open-MOPD identifies token count, reward scale, and stale rewards as sources of unequal update budgets (Gao et al., 2026). D^3 -MOPD uses teacher-loss gap and descent velocity to update the domain mixture (Sun et al., 2026). Teacher mixtures can also create cross-domain capability transfer and a mixture-dependent seesaw (Li et al., 2026). The cross-domain transfer results motivate our use of teacher loss for objective comparisons and separate benchmark reporting for capability.

Sampling with optimizer state and cost. Gradient-norm importance sampling reduces the second moment of an SGD estimate (Alain et al., 2015; Katharopoulos & Fleuret, 2018). Cost-aware sampling adds a square-root cost correction and provides convergence guarantees beyond convex objectives (Mohri et al., 2026). The same allocation shape appears in classical Neyman allocation (Neyman, 1934; Cochran, 1977). Adaptive optimizers create an extra interaction because the sampling weight is squared in the second-moment state (Lahire, 2023). GPAS uses an AdamW metric whose population target remains fixed under proposal changes.

3 OBJECTIVE AND SAMPLING PROTOCOL

There are M tasks. Task i has loss L_i and target weight $\lambda_i \geq 0$, with $\sum_i \lambda_i = 1$. The target is Equation 1. The weights specify a mixture of teacher losses. Downstream capabilities can respond to several teacher losses, so we evaluate capabilities separately (Li et al., 2026).

Each step selects one task, generates a task-homogeneous batch, and produces a stochastic gradient G_i with mean $h_i = \nabla L_i(\theta)$. Our MOPD batches use a fixed valid-token budget. We average tokens inside each response and then average responses. The logs report the realized response count for every task. Token averaging inside a task is also compatible with the analysis because the sampling result only requires a fixed definition of G_i .

For MOPD, L_i is the per-step teacher-loss surrogate on student-sampled prefixes. Rollout sampling sits outside backpropagation. Proposition 1 applies to the resulting task-gradient estimator at each step.

The one-task regime applies when a step uses one resident teacher or one reward service and generation dominates its cost. A mixed batch is a different execution regime. Token-share weighting, as used by Open-MOPD, is a suitable way to specify task weights in that setting (Gao et al., 2026).

Let H_t contain the model, optimizer state, and all observations before task I_t is drawn. The target λ_t and proposal q_t may depend on H_t . Write $h_{t,i} = \mathbb{E}[G_{t,i} \mid H_t]$. After drawing $I_t \sim q_t$, define

$$w_t = \frac{\lambda_{t,I_t}}{q_{t,I_t}}, \quad \hat{g}_t = w_t G_{t,I_t}. \quad (2)$$

We require $q_{t,i} > 0$ for every task with $\lambda_{t,i} > 0$.

Proposition 1 (Objective preservation through AdamW moments). *Conditioned on H_t ,*

$$\mathbb{E}[\hat{g}_t \mid H_t] = \sum_i \lambda_{t,i} h_{t,i}. \quad (3)$$

Use $\hat{g}_t$ in AdamW’s first moment and use $w_t G_{t,I_t}^2$ in its second moment, where the square is element-wise:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) w_t G_{t,I_t}, \quad (4)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) w_t G_{t,I_t}^2. \quad (5)$$

The expected changes in m_t and v_t equal those of the baseline sampler $q_t = \lambda_t$, conditioned on the same history.

Proof. Summing $q_{t,i}(\lambda_{t,i}/q_{t,i})h_{t,i}$ gives Equation 3. Replacing $h_{t,i}$ with $\mathbb{E}[G_{t,i}^2 \mid H_t]$ gives the matching second-moment observation. Substitution in the two recursions gives the stated moment changes. $\square$

We call Equation 5 moment-consistent AdamW. Bias correction, weight decay, and the parameter update remain unchanged. The implementation changes one line in the second-moment update.

Remark 1 (Why the AdamW change is needed). Standard AdamW uses $\hat{g}_t^2$ in its second moment. Its conditional mean observation is

$$\sum_i \frac{\lambda_{t,i}^2}{q_{t,i}} \mathbb{E}[G_{t,i}^2 \mid H_t], \quad (6)$$

which changes with q_t and can partially absorb the sampling correction through AdamW’s denominator (Lahire, 2023). The once-weighted update gives the optimizer metric a proposal-independent population target. Section 5 measures this difference.

4 OPTIMIZER-AWARE TASK ALLOCATION

We condition on the state before task selection and omit the step index. Let A apply the optimizer scale stored at that point. For SGD, A is the identity. For AdamW, we use the lagged preconditioner

$$AG_i = \frac{G_i}{\sqrt{\bar{v}} + \epsilon}, \quad s_i^2 = \mathbb{E}\|AG_i\|_2^2, \quad (7)$$

where $\bar{v}$ is the bias-corrected second moment before the draw. The score s_i measures the scale of task i in the coordinates used by AdamW.

Proposition 2 (Optimizer-aware proposal). *Let $h = \sum_i \lambda_i h_i$. The mean squared error of one corrected draw is*

$$M(q) = \mathbb{E} \|A(\hat{g} - h)\|_2^2 = \sum_i \frac{\lambda_i^2 s_i^2}{q_i} - \|Ah\|_2^2. \quad (8)$$

Its minimum is attained by

$$q_i^* = \frac{\lambda_i s_i}{\sum_j \lambda_j s_j}. \quad (9)$$

Proof. Cauchy–Schwarz gives $\sum_i \lambda_i^2 s_i^2 / q_i \geq (\sum_i \lambda_i s_i)^2$. Equality requires q_i proportional to $\lambda_i s_i$. The second term in Equation 8 is fixed before the draw. $\square$

Corollary 1 (Cost-aware proposal). *Let c_i be the measured time for one task- i step. The proposal minimizing*

$$J(q) = \left(\sum_i q_i c_i \right) \left(\sum_i \frac{\lambda_i^2 s_i^2}{q_i} \right) \quad (10)$$

is

$$q_i^* \propto \frac{\lambda_i s_i}{\sqrt{c_i}}. \quad (11)$$

For the corollary, Cauchy–Schwarz gives $J(q) \geq (\sum_i \lambda_i s_i \sqrt{c_i})^2$, with equality at Equation 11. The resulting proposal is the cost-constrained Neyman rule with tasks as strata (Neyman, 1934; Cochran, 1977; Mohri et al., 2026). Mohri et al. also justify the uncentered objective in Equation 10 through convergence rates for convex and non-convex optimization; Appendix F of their paper discusses the scope of that connection.

For uniform target weights, the second-moment ratio has a simple range:

$$\frac{(\sum_i s_i)^2}{M \sum_i s_i^2} \in [1/M, 1]. \quad (12)$$

With four tasks, score heterogeneity permits a speedup of up to four in this local measure. Equal scores recover uniform sampling.

Online estimates. After a task- i step, GPAS updates

$$\hat{s}_i^2 \leftarrow \gamma \hat{s}_i^2 + (1 - \gamma) \|AG_i\|_2^2. \quad (13)$$

Cost-GPAS maintains the same moving average for runtime. A short round-robin warm start initializes every task. A probability floor limits importance weights and refreshes each task estimate regularly. The sampler needs one score and one optional runtime estimate per task.

Remark 2 (Measuring score error). For the cost rule, define the cost-biased probability $\tilde{q}_i = q_i c_i / (\sum_j q_j c_j)$. Theorem 4.2 of Mohri et al. (2026) gives the exact diagnostic

$$\frac{J(\hat{q})}{J(q^*)} = 1 + D_{\chi^2}(\tilde{q}^* \parallel \tilde{q}). \quad (14)$$

The frozen-gradient bank computes the right side from an all-task score snapshot. During training, periodic all-task refreshes define the reference q^* , the online moving averages define $\hat{q}$, and the log records the reference age.

5 EXPERIMENTS

The evaluation has three layers. A controlled problem checks the two propositions. Gradients cached from one Uniform run measure the mechanism on an LLM. Matched MOPD runs measure weighted teacher loss per token and GPU hour. Each measurement corresponds to one part of the method.

5.1 CONTROLLED VALIDATION

We construct three Gaussian task-gradient distributions with different raw scales, AdamW coordinate scales, and task costs. Every proposal targets $\lambda_i = 1/3$ and uses the factor in Equation 2. The estimator experiment uses 20,000 Monte Carlo trials and 16 corrected task gradients per trial.

Under the identity metric, Gradient-Norm IS reaches a calculated and measured error ratio of 0.534 relative to Uniform. In the AdamW metric, GPAS reaches 0.698 by calculation and 0.697 by simulation. The raw-norm proposal reaches 7.178 times the Uniform error because its largest raw coordinates receive a small AdamW scale. Cost-GPAS reaches 0.893 of Uniform on $J(q)$.

The fourth panel draws tasks from a proposal different from λ . Both optimizers update the first moment with wG_I . Standard AdamW updates the second moment with $w^2G_I^2$, while moment-consistent AdamW uses wG_I^2 . We report each expected moment change relative to the baseline $q = \lambda$. The moment-bias comparison directly checks Proposition 1. With one million draws, both methods have first-moment bias 0.008. Standard AdamW has second-moment bias 0.816, close to the calculated value 0.818. Moment-consistent AdamW has measured bias 0.002, consistent with Monte Carlo error around zero.

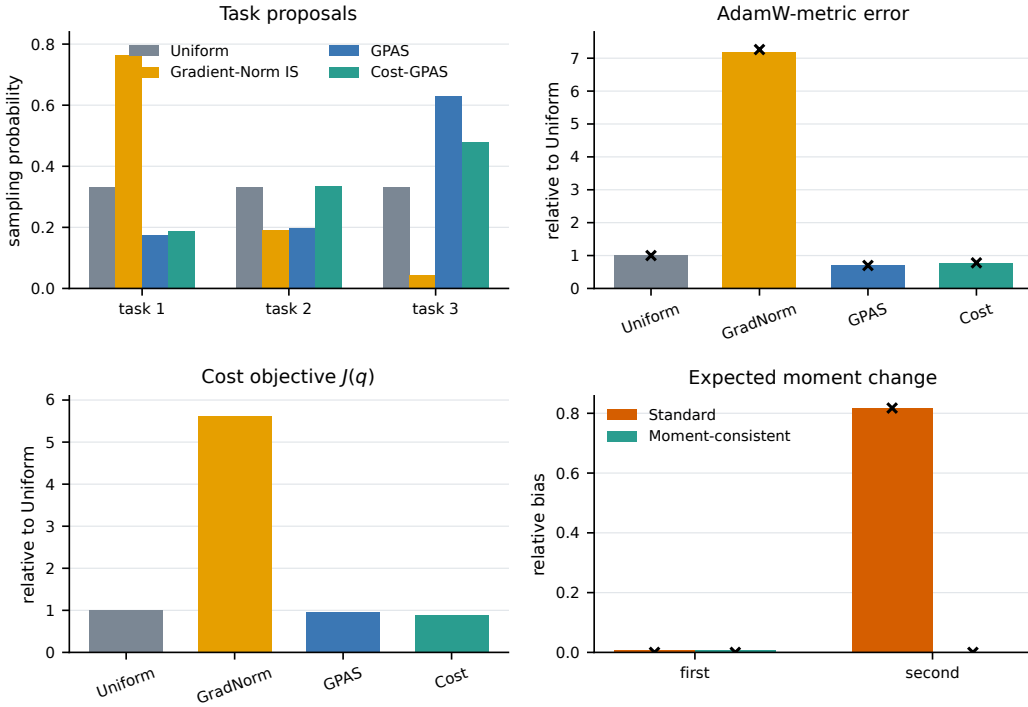


Figure 1: Controlled validation. The first three panels compare task proposals, AdamW-metric estimator error, and the cost objective $J(q)$. Black markers show calculated values. The fourth panel reports relative bias in the expected AdamW moment changes for $q \neq \lambda$. Lower values are better in the last three panels.

5.2 FROZEN-GRADIENT MECHANISM STUDY

The Uniform run supplies checkpoints at the start, middle, and end of training. At each checkpoint, we collect 16 task batches per task. Collection stores sufficient statistics in place of full gradient copies whenever a scalar or fixed projection is enough.

The study answers four questions. First, Pearson and Spearman correlations compare raw gradient norm and AdamW-metric norm with the task-dependent part of the next AdamW update. Second, the cached gradients give $M(q)$, $J(q)$, and the χ^2 gap in Equation 14 for every proposal. Third, gradient replay measures proposal-dependent drift in v under standard and moment-consistent AdamW.

Fourth, pairwise cosine similarity of task mean gradients separates direction differences from scale differences. The cosine matrix appears as one subpanel of the mechanism figure.

These measurements use the same saved model and optimizer state for every proposal. They require one training run and add backward passes only at the three saved checkpoints.

5.3 FOUR-TEACHER MOPD STUDY

The student is a fixed revision of Qwen3-1.7B (Qwen Team, 2025). Four same-origin teachers specialize in mathematics, code, instruction following, and science question answering. The paper reports the model and tokenizer revisions, teacher checkpoints, and each teacher’s score on its own domain.

Each optimizer step samples one task and uses one task-homogeneous batch. A batch targets 65,536 valid response tokens; the logs also report the realized number of responses for each task. Every response is averaged over its valid tokens, followed by an average over responses. A shared norm clip is applied to G_i before sampling correction. The optimizer receives G_i and w separately, uses wG_i for m , and uses wG_i^2 for v .

Runs use an 8,192-token response cap, temperature 1, top- p 1, an eight-step round-robin warm start, $\gamma = 0.95$, and $q_{\min} = 0.05$. A 300-step pilot uses one paired seed for Uniform and GPAS. The main comparison uses 400 steps, with the pilot checkpoints reused. The five primary fixed-target methods use three paired seeds; the moment ablation uses two. Methods within a seed share initialization, per-task data order, and evaluation prompts. Teacher loss is evaluated every 50 steps, and capability benchmarks are evaluated at the endpoint.

5.4 METHODS AND DECOMPOSITION

Table 1 separates fixed-target allocation from curriculum target changes. The first five rows use $\lambda_i = 1/4$ and moment-consistent AdamW. Gradient-Norm IS uses the identity metric. D³-Proposal-IC uses the D³ loss-gap and descent-velocity signal only as q , followed by importance correction. Its target stays fixed.

Table 1: MOPD comparisons. The eight rows require 21 runs.

Method	Target λ	Proposal q	Seeds
Uniform	1/4	1/4	3
Gradient-Norm IS	1/4	$\propto \lambda \hat{r}$	3
GPAS	1/4	$\propto \lambda \hat{s}$	3
Cost-GPAS	1/4	$\propto \lambda \hat{s} / \sqrt{\hat{c}}$	3
D ³ -Proposal-IC	1/4	D ³ signal + correction	3
Native D ³	q_t	D ³ signal	2
D ³ target + GPAS	replayed λ_t	$\propto \lambda_t \hat{s}$	2
GPAS + standard AdamW	1/4	$\propto \lambda \hat{s}$	2

Native D³ uses its proposal as its target and applies a unit gradient factor. For each seed, the D³ target + GPAS run replays the exact λ_t sequence recorded by the Native D³ run. This pairing fixes the realized target path and changes only task allocation. The last row uses the same GPAS proposal rule and wG_i first moment as GPAS, with $w^2G_i^2$ in the second moment.

5.5 METRICS AND COMPARISONS

The primary metric is weighted held-out teacher loss

$$L(N) = \sum_i \lambda_i \ell_i(N), \quad (15)$$

plotted against cumulative valid response tokens and measured GPU hours. We report normalized loss AUC on the common resource interval, final weighted loss, and every per-task loss. Mean curves and standard errors use the three paired seeds for the five fixed-target methods.

The fixed-target rows compare allocation quality under one teacher-loss objective. Gradient-Norm IS versus GPAS isolates the optimizer metric. D³-Proposal-IC versus GPAS compares proposal signals. GPAS versus its standard AdamW ablation isolates the moment correction. Native D³ versus D³ target + GPAS shares the same target sequence and isolates allocation on top of curriculum weights.

Capability evaluation is secondary. It reports MATH-500 greedy pass@1, a frozen post-cutoff Live-CodeBench pass@1 slice, IFBench strict accuracy, and GPQA-Diamond average@4. The native D³ rows use per-task losses and capability scores because their target changes over time. Every run logs task identity, q_i , λ_i/q_i , valid tokens, response count, GPU time, raw and AdamW-metric gradient norms, score age, and teacher loss.

6 DISCUSSION AND CONCLUSION

GPAS separates the weighted teacher-loss target from compute allocation. Moment-consistent AdamW keeps the expected first- and second-moment changes equal to the baseline $q = \lambda$. GPAS then allocates task draws using gradient scale in AdamW’s lagged metric, and Cost-GPAS also accounts for task time. The identity metric recovers gradient-norm importance sampling.

The controlled estimator study supports the local allocation result. GPAS reduces AdamW-metric error by 30.3%, while raw gradient-norm sampling performs poorly in the constructed AdamW geometry. The moment-state experiment checks the one-line AdamW change directly. Frozen-gradient replay applies the same measurements to LLM gradients. The matched MOPD design measures the connection between local error and teacher-loss efficiency.

Three boundaries guide interpretation. First, matching expected moment changes still allows different parameter paths because task order changes the realized gradients. Low-discrepancy task schedules may reduce this path dependence and are a useful direction for future work. Second, GPAS uses recent gradients and a lagged optimizer state. Score-age logs and frozen-gradient replay measure the quality of this approximation. Third, Cost-GPAS minimizes the uncentered second-moment objective in Equation 10. Appendix F of Mohri et al. (2026) connects this objective to optimization rates, while GPU-hour curves measure its practical value.

The main design principle is simple: choose the teacher-loss objective first, then allocate expensive task steps for that objective. The same separation lets GPAS compose with a curriculum by applying the allocation rule after each target update.

AI USE STATEMENT

We used generative AI tools for language editing and manuscript organization. The authors reviewed every AI-assisted revision and take responsibility for the final text, claims, proofs, code, and experimental results.

ETHICS STATEMENT

The study evaluates optimization methods for existing language models with established reasoning and instruction-following datasets and automated verifiers. Model and dataset biases can affect the reported capability scores. We record data provenance, invalid outputs, and verifier failures to support careful interpretation of the results.

REPRODUCIBILITY STATEMENT

Section 5 specifies the model family, task setup, update budget, evaluation points, and evaluation protocol. The appendix gives the sampling derivations and implementation details. The repository includes the AdamW moment experiment code, numerical results for both controlled studies, and figure-generation code. Each reported large-model result is accompanied by its data manifest, model and tokenizer revisions, optimizer settings, task-sampling log, saved model state, and per-prompt evaluation output.

REFERENCES

- Guillaume Alain, Alex Lamb, Chinnadurai Sankar, Aaron Courville, and Yoshua Bengio. Variance reduction in sgd by distributed importance sampling, 2015. URL <https://arxiv.org/abs/1511.06481>.
- Zhao Chen, Vijay Badrinarayanan, Chen-Yu Lee, and Andrew Rabinovich. Gradnorm: Gradient normalization for adaptive loss balancing in deep multitask networks. In *Proceedings of the 35th International Conference on Machine Learning*, pp. 794–803, 2018. URL <https://proceedings.mlr.press/v80/chen18a.html>.
- William G. Cochran. *Sampling Techniques*. Wiley, 3 edition, 1977.
- Nicholas E. Corrado, Wenyuan Huang, and Josiah P. Hanna. Distributionally robust multi-task reinforcement learning via adaptive task sampling, 2026. URL <https://arxiv.org/abs/2605.14350>.
- Huan-ang Gao, Haohan Chi, Yong Yan, Shiyuan Feng, Hanlin Wu, Zheng Jiang, Bingxiang He, Wei-Ying Ma, Ya-Qin Zhang, and Hao Zhou. Open-mopd: Diagnosing and fixing capability imbalance in multi-teacher on-policy distillation, 2026. URL <https://arxiv.org/abs/2608.19098>.
- Angelos Katharopoulos and François Fleuret. Not all samples are created equal: Deep learning with importance sampling. In *Proceedings of the 35th International Conference on Machine Learning*, pp. 2525–2534, 2018. URL <https://proceedings.mlr.press/v80/katharopoulos18a.html>.
- Thibault Lahire. Non-uniform sampling and adaptive optimizers in deep learning. OPT 2023: Optimization for Machine Learning, 2023. URL <https://opt-ml.org/papers/2023/paper20.pdf>.
- Zhaoyi Li, Deyang Kong, Yuan Wei, Evan Yang, Ranran Shen, Mahardika Krisna Ihsani, Ming Yang, Wei Zhang, Chuan Hao, Jian Yang, Ran Tao, Bryan Dai, Shikun Zhang, Wei Ye, Ying Wei, and Defu Lian. Every coin has two sides: On the dual nature of generalization in on-policy distillation of large language models, 2026. URL <https://arxiv.org/abs/2608.16647>.
- Bo Liu, Xingchao Liu, Xiaojie Jin, Peter Stone, and Qiang Liu. Conflict-averse gradient descent for multi-task learning, 2024. URL <https://arxiv.org/abs/2110.14048>.
- Wenhan Ma, Jianyu Wei, Liang Zhao, Hailin Zhang, Bangjun Xiao, Lei Li, Qibin Yang, Bofei Gao, Yudong Wang, Rang Li, Jinhao Dong, Zhifang Sui, and Fuli Luo. Mopd: Multi-teacher on-policy distillation for capability integration in llm post-training, 2026. URL <https://arxiv.org/abs/2606.30406>.
- Clara Mohri, Amir Globerson, Haim Kaplan, Tomer Koren, and Yishay Mansour. Cost-aware learning, 2026. URL <https://arxiv.org/abs/2604.28020>.
- Jerzy Neyman. On the two different aspects of the representative method: The method of stratified sampling and the method of purposive selection. *Journal of the Royal Statistical Society*, 97(4): 558–625, 1934. doi: 10.2307/2342192.
- Qwen Team. Qwen3 technical report, 2025. URL <https://arxiv.org/abs/2505.09388>.
- Ozan Sener and Vladlen Koltun. Multi-task learning as multi-objective optimization, 2019. URL <https://arxiv.org/abs/1810.04650>.
- Zechen Sun, Zhiwei Zhang, Fei Zhao, Juntao Li, Mu Chuan, Huayu Deng, Guojian Zhan, Wenliang Chen, Yao Hu, and Min Zhang. D³-mopd: Adaptive dynamic domain scheduling for efficient multi-teacher distillation, 2026. URL <https://arxiv.org/abs/2608.24987>.
- Tianhe Yu, Saurabh Kumar, Abhishek Gupta, Sergey Levine, Karol Hausman, and Chelsea Finn. Gradient surgery for multi-task learning, 2020. URL <https://arxiv.org/abs/2001.06782>.

A PRACTICAL TRAINING RECIPE

The recipe uses one task-homogeneous batch and one optimizer step after each rollout. Task selection occurs before generation, so a step calls one teacher or reward service.

1. Fix the teacher-loss target. Choose λ for the weighted teacher-loss objective and record it before choosing a sampler. Benchmark evaluation measures the resulting capability profile separately.

2. Define one task gradient. Fix the token and response aggregation rule inside each task. Our MOPD setup averages tokens inside each response and then averages responses. It retains the existing fixed-token batch budget and records the realized response count, prompt count, valid-token count, and GPU time for each task.

3. Initialize the sampler. Run a short round-robin warm start. For each task, store a moving average of raw gradient norm and AdamW-metric norm. Cost-GPAS also stores a moving average of task time. The four-task protocol uses eight warm-start steps and $\gamma = 0.95$.

4. Keep the raw gradient and weight separate. After sampling task I , compute its raw task gradient G_I and the factor $w = \lambda_I/q_I$. Moment-consistent AdamW uses wG_I for the first moment and wG_I^2 for the second moment. Passing only wG_I to an unchanged AdamW implementation produces the standard $w^2G_I^2$ second moment. The optimizer interface therefore accepts both G_I and w .

5. Update and save the sampler. GPAS uses $q_i \propto \lambda_i \hat{s}_i$; Cost-GPAS uses $q_i \propto \lambda_i \hat{s}_i / \sqrt{\hat{c}_i}$. Apply the probability floor before drawing a task. Save score and cost averages with model and optimizer state. Log I , q_I , w , valid tokens, response count, GPU time, raw norm, AdamW-metric norm, score age, and clipping status.

B GRADIENT AND EVALUATION DETAILS

B.1 SAMPLED-TOKEN MOPD GRADIENT

Fix a student-generated prefix z . Write $p_v = \pi_\theta(v | z)$ and $\tau_v = T_i(v | z)$. The reverse-KL ascent gradient at that prefix is

$$u_i(z) = \sum_v p_v \log \frac{\tau_v}{p_v} \nabla_\theta \log p_v. \quad (16)$$

For $V \sim p$, the sampled-token update

$$U_i(z, V) = \text{sg} \left[\log \frac{\tau_V}{p_V} \right] \nabla_\theta \log p_V \quad (17)$$

has conditional mean $u_i(z)$. The implementation averages these token updates inside each response and then averages responses to form G_i .

B.2 FROZEN-GRADIENT EVALUATION METRICS

At one checkpoint, the gradient bank estimates each task mean h_i , raw second moment, AdamW-metric second moment s_i^2 , and runtime c_i . For every proposal, it reports $M(q)$ from Equation 8 and $J(q)$ from Equation 10, both relative to Uniform.

To evaluate the metric, each cached G_i is inserted into the saved AdamW state while all other quantities are held fixed. Pearson and Spearman correlations compare the resulting update norm with $\|G_i\|_2$ and $\|AG_i\|_2$. Pairwise cosine similarity uses task mean gradients at the same checkpoint. A fixed random projection can replace full mean-gradient storage; the projection seed and dimension are shared by all tasks.

For the state replay, a task sequence is drawn from each proposal. Standard AdamW accumulates $w^2G_i^2$, and moment-consistent AdamW accumulates wG_i^2 . The reported drift is the distance from the state produced by the baseline $q = \lambda$, normalized by the baseline state norm.

B.3 EXACT SCORE-ERROR DIAGNOSTIC

Let $B = \sum_i \lambda_i s_i \sqrt{c_i}$. The optimal cost-aware proposal has $J(q^*) = B^2$. For the cost-biased probabilities in Equation 14, direct substitution gives

$$1 + D_{\chi^2}(\tilde{q}^* \parallel \tilde{q}) = \sum_i \frac{(\tilde{q}_i^*)^2}{\tilde{q}_i} = \frac{J(\hat{q})}{B^2} = \frac{J(\hat{q})}{J(q^*)}. \quad (18)$$

The frozen bank estimates q^* from all tasks at one saved state. During training, periodic full-task refreshes provide the reference scores; the log records the divergence, the reference age, and the contribution of the probability floor.

B.4 STRICT CURRICULUM DECOMPOSITION

An online curriculum can generate different target sequences after a proposal changes the training path. Our paired decomposition fixes this source of variation. Native D³ first records its target sequence $\lambda_1, \dots, \lambda_T$ for each seed. The paired D³ target + GPAS run replays that sequence and changes only q_t . Both runs share initialization, per-task data order, and evaluation prompts.

D³-Proposal-IC serves a separate purpose. It keeps $\lambda_i = 1/4$, forms q_t from the published loss-gap and descent-velocity signal, and applies $\lambda_i/q_{t,i}$. Its comparison with GPAS measures proposal quality under one fixed teacher-loss target.

B.5 SAMPLER STATE AND OPTIMIZER INTERFACE

Each checkpoint stores the score average, runtime average, observation count, score age, probability floor, and current proposal for every task. The optimizer checkpoint stores the usual AdamW state and a flag identifying the second-moment rule. Resume tests compare the next proposal and the next optimizer state before and after serialization.

The main theory uses the task gradient before norm clipping. The training code uses a shared emergency clip before importance correction and reports the clip rate by task and method. A triggered clip defines a clipped update field for that step. The fixed-objective interpretation is reported together with this rate.